Agents Report, Coordinators Decide: A Governance-First Multi-Agent Orchestration Architecture with Claim-Level Conflict Reconciliation and Human-Gated Memory

Draft v1 · October 2026 · [Author names] · [Affiliation] · [Contact]

> **Status note (delete before submission).** Sections 3–5 describe the system as documented in the public repository (README, ARCHITECTURE.md and merged pull request #3). The source files were not independently audited for this draft, so every technical claim must be checked against the code. Section 6 is an evaluation protocol: no quantitative results exist yet, and every [TBD] must be replaced with measured values. References marked [verify] were cited from memory and must be checked before submission.

Abstract

Multi-agent LLM systems are attractive for ambiguous enterprise goals, yet published failure analyses attribute many breakdowns to system design, inter-agent misalignment and weak verification rather than to model capability alone [6]. We present AI-OS, an open-source orchestration platform built on LangGraph that turns three behaviours usually buried in prompts into explicit, testable policy: how disagreement between agents is detected and reconciled, how tool failure is handled through bounded retry, degradation or replanning, and how findings become durable knowledge. The central design rule is that agents report and the coordinator decides. Agents return structured claims (claims, confidence, evidence) instead of prose. A pure scheduling function with no I/O compares claims on a (subject, metric) key, re-queries the least confident claimant when claims contradict, and escalates unresolved contradictions to a human. A human sign-off gate, enforced by a graph checkpoint, precedes memory promotion; promoted facts carry provenance, and contradicted facts are superseded rather than overwritten. Every plan, dispatch, tool call, retry, conflict and promotion is recorded as a traced event. We describe the architecture, document two scenarios, and state its known weaknesses: literal conflict matching and uncalibrated self-reported confidence. We then give a pre-registered evaluation protocol with baselines, ablations and metrics. [TBD: one-sentence summary of headline results, e.g. reconciliation accuracy, silent-error rate and cost overhead versus baselines.]

Keywords: multi-agent systems, LLM agents, orchestration, conflict resolution, fault tolerance, agent memory, human-in-the-loop, observability

1. Introduction

Enterprise questions rarely arrive as well-formed tasks. A request such as ‘report SLA breaches for the weekly review’ requires decomposing a goal, pulling facts from sources that may disagree, coping with services that go down, and deciding what the organisation should remember. Multi-agent frameworks such as AutoGen [3] and graph runtimes such as LangGraph [4] supply the plumbing but leave policy decisions to the developer. A large empirical study annotated more than 1,600 execution traces from seven multi-agent frameworks and organised 14 failure modes into three categories: system-design issues, inter-agent misalignment and task verification [6]. Those categories describe exactly the places where a framework’s defaults matter.

This paper argues that three production-critical decisions should be lifted out of prompts and made explicit, inspectable policy: reconciliation, failure handling and memory promotion. We describe AI-OS, a platform that implements this view, and make the following contributions.

1. **Policy/agent separation.** Agents never decide what to do about a failure or a contradiction; they report. A pure, I/O-free coordinator function takes run state and returns a decision, so scheduling rules are unit-testable independently of any model or framework.
2. **Claim-level reconciliation.** Agents emit structured claims keyed by (subject, metric). The coordinator detects cross-agent contradictions, re-queries the least confident claimant with the contradiction as context, and escalates to a human if the contradiction survives.
3. **Gated, supersede-only memory.** Promotion to durable memory requires human approval and a confidence bar. A contradicted fact is superseded, never overwritten, and every fact keeps provenance to the run that produced it.
4. **Bounded failure handling with disclosure.** Failed tools are retried with backoff. Non-critical tasks degrade and the gap is named in the report; blocked critical tasks trigger replanning.
5. **Traceability.** Every decision is a typed, queryable trace event, supporting post-hoc diagnosis in the spirit of failure taxonomies such as MAST [6].
6. **An evaluation protocol** (Section 6) that turns each design claim into a falsifiable hypothesis with baselines and ablations. Results are pending in this version.

A note on naming. The name AI-OS invites comparison with AIOS from Rutgers [1], an agent operating system whose kernel provides scheduling, context management, memory management, storage management and access control for concurrent LLM agents. Our system does not manage LLM serving resources. It is better described as a governance layer for agent orchestration; Section 7 discusses where the operating-system analogy holds and where it does not.

2. Background and Related Work

Agent operating systems and OS-inspired memory. AIOS [1] isolates LLM-specific services into a kernel with scheduling, context switching, memory and storage management and access control, and reports up to 2.1× faster execution for agents built with different frameworks. MemGPT [2] borrows virtual-memory paging to manage limited context windows through function calls. A survey of AI–OS integration [21] covers the wider landscape. These works concern resource management for the model; AI-OS concerns the correctness and governance of what agents collectively conclude.

Multi-agent frameworks. AutoGen [3] programs agents as conversable entities coordinated through conversation patterns such as group chat, with human participation through proxy agents. LangGraph [4] models workflows as stateful graphs with persistence and node-level interrupts for human approval. AI-OS adopts the graph substrate because checkpointed interrupts make a sign-off gate a first-class state, and layers policy on top.

Task-graph planning. HuggingGPT [22] uses an LLM as controller to decompose requests into sub-tasks. LLMCompiler [5] has a planner emit a DAG of function calls with dependencies so that independent calls run in parallel, and reports speedups over sequential ReAct-style calling [23]. AI-OS also plans a DAG, but treats structural validity as a gate: invalid graphs are rejected and the planner is re-prompted with the reason, and a blocked critical task triggers replanning.

Failures, fault tolerance and reliability. MAST [6] provides an empirically grounded failure taxonomy. Recent benchmarks inject tool faults: ToolMaze [14] reports that implicit semantic failures reduce recovery sharply relative to explicit errors; Failing Tools [18] injects availability denial, data staleness, silent no-ops and corrupted state; AgentChaosBench [15] targets diagnosing fault types from telemetry; AgentProp-Bench [17] studies how injected errors propagate; and a study of self-healing orchestrators [16] compares orchestration-level recovery policies under controlled fault injection. These are recent preprints and their final venues should be rechecked. τ-bench [10] introduced pass^k to measure consistency over repeated trials, and a survey [20] argues that robustness and reliability need dedicated evaluation. We reuse these ideas to evaluate AI-OS's policies.

Knowledge conflicts and confidence. Xu et al. [9] categorise conflicts as context–memory, inter-context and intra-memory. Cross-agent contradictions in AI-OS are an instance of inter-context conflict, and reviews of that survey report that LLMs are often poor at detecting such contradictions [verify against the survey text]. Re-query and escalation rules depend on confidence, yet verbalised confidence is known to be overconfident, and consistency-based and hybrid estimators were found to outperform purely verbalised ones in most cases [7]. Kadavath et al. [8] studied calibration of model self-evaluation.

Agent memory. Mem0 [12] extracts, consolidates and retrieves salient facts; as characterised in [19], its update and delete operations can discard history. Zep [13] builds a temporal knowledge graph in which contradicted edges are invalidated rather than deleted. Newer provenance-oriented designs retain displaced values with their sources [19, 24]. The security literature argues for treating each memory write as a privileged state transition, with provenance and authorisation checks before commit, and names human or policy arbitration as a defence [11]. AI-OS's promotion gate and supersede-only rule are a simple, concrete instance of this principle.

Positioning. Each ingredient has precedent. Our contribution is their integration under one separation principle (agents report, coordinator decides), an explicit and testable policy surface, and, once Section 6 is executed, evidence about which policies matter and what they cost.

3. System Design

3.1 Design principles

- **P1. Agents report; the coordinator decides.** An agent that hits a failing tool returns an error report; it does not choose to retry.
- **P2. Claims, not prose.** Agent outputs are structured so that they can be compared mechanically.
- **P3. Pure policy.** The scheduling policy imports no graph framework and performs no I/O.
- **P4. Everything is an event.** Each decision lands in a trace that can be queried afterwards.
- **P5. Engine separate from hosting.** According to the repository, the coordinator, run-state, agent-to-agent envelope and semantic-memory files have not changed since the initial commit, while platform, local-model and own-data layers were added around them. This is offered as evidence of separation and should be verified with the commit history.

3.2 Run lifecycle

1. **Plan.** The planner asks the model for a task DAG and validates it (Section 3.3).
2. **Dispatch.** Every task whose dependencies have resolved is dispatched in parallel using LangGraph's Send primitive, one branch per task, merged through a state reducer.
3. **Execute.** Each agent works in two phases (Section 3.4).
4. **Reconcile.** The coordinator compares claims across agents (Section 3.5).
5. **Handle failure.** A failed tool is retried with backoff; then the task degrades if non-critical, or blocks and triggers a replan if critical.
6. **Gate.** The graph interrupts at a sign-off node and checkpoints. A paused run holds no worker and can be resumed from another process, on another day.
7. **Promote.** On approval, claims that clear the confidence bar become durable facts with provenance and inform the next run's plan.

3.3 Planner and plan validity

Let a plan be a graph G = (T, E) of tasks T and dependency edges E, where each task t has an assigned agent a(t) and a criticality flag. The planner accepts G only if V(G) holds: G is acyclic; every dependency refers to a task in T; every a(t) is a registered agent; and G contains a terminal reporting task. If V(G) fails, the planner is re-prompted with the violated condition. When a critical task is blocked at run time, the planner is invoked again to produce a revised graph. Validation is structural; it does not assess whether the decomposition is semantically good, which is evaluated in Section 6 (RQ1).

3.4 Two-phase agent contract

In phase one the model selects tool calls from the tools granted to that agent. A gateway authorises each call, enforces timeouts, executes it and traces it. In phase two the model converts tool results into a structured set of claims. A claim is a tuple c = (s, m, v, p, E): subject s, metric m, value v, self-reported confidence p in [0, 1], and evidence references E. Agents exchange messages through a typed envelope.

3.5 Coordinator and reconciliation

The coordinator is a function Decide(state) → action. Its policies are:

- **Dispatch:** release all ready tasks.
- **Retry:** bounded retries with backoff on tool errors.
- **Degrade or block:** non-critical tasks degrade and the gap is named in the final report; critical tasks block and trigger replanning.

- **Reconcile:** two claims c_i and c_j from different tasks conflict iff (s_i, m_i) = (s_j, m_j) and v_i ≠ v_j, with exact comparison. On conflict, the claimant with the lowest p is re-queried with the contradiction supplied as context. If the contradiction survives, it is escalated to a human.

Because Decide is pure, each rule can be tested with synthetic state. The weakness of the design is also visible here: identity and equality are literal, so 55,700 and 55700.00 do not match, and p is uncalibrated yet drives targeting, escalation and promotion.

3.6 Memory and promotion

Memory has two tiers: episodic memory (graph checkpoints) and durable memory (facts). A claim c from a run is promoted iff the run was approved by a human and p(c) ⩾ θ, where θ is the confidence bar [TBD: value in code]. If c contradicts a stored fact f, then f is marked superseded and retained, and c is stored with provenance to the originating run and evidence. Recall combines lexical matching with recency and confidence; the repository states this is adequate at hundreds of facts and poor at tens of thousands, and identifies the recall function as the replacement seam.

3.7 Tool gateway, grants and own-data safety

The gateway authorises every tool call against per-tenant grants and records it in the trace. Pull request #3 added domain packs (a TOML file describing data sources, permitted questions, searchable documents and agent descriptions) and a folder-of-files source: spreadsheets are ingested into a throwaway SQLite copy with one table per file or sheet, and documents (Markdown, text, PDF, Word) are read as text. Three safety decisions are documented. First, databases are opened read-only at the driver, not by convention. Second, named queries are the default: the model selects a query by name and supplies parameters but never writes SQL. Third, ad-hoc SELECT is allowed only against an ingested copy, is refused for live databases, and is limited to one SELECT statement capped at 500 rows. Tenant grants are now derived from the tools a domain actually publishes, fixing a bug in which domain-specific tools were registered and then silently dropped. Three grant fields (rate limit per minute, secret reference, maximum wall-clock seconds) are stored but not yet enforced.

3.8 Observability

Every plan, dispatch, tool call, retry, conflict, resolution, sign-off request and promotion is a traced event with a sequence number, stored as a per-run trace file and renderable as an SVG. Because a run can be suspended and resumed in another process, the trace is designed to remain a single ordered record across the pause.

3.9 Optional hosting platform and local models

About one third of the code base supports multi-user hosting: queue, leases, workers, tenancy, budgets, a review inbox, an API and a console. Model clients cover hosted models, self-hosted inference (vLLM, Ollama, llama.cpp) and an offline replay client that returns recorded model responses while the engine, tools and memory run for real.

4. Documented Scenarios

4.1 Support SLA review. Priority-1 tickets must close within seven days, and any team with three or more overdue tickets is reported to the director. The live export shows eight overdue tickets; last Monday's status report shows four. The two sources disagree on a value that decides whether the director is informed. The coordinator detects the conflict, re-queries the less confident source, and learns that the report is a Monday 08:00 snapshot taken before more tickets aged out. The resolution and its reason are recorded, and the run stops at the sign-off gate. After approval, Payments, which holds three of the eight overdue tickets, is escalated.

4.2 Audit review. A second scenario adds a flaky tool that recovers on retry, an unreachable tool that causes a non-critical task to degrade with the gap named in the report, and a variant in which a dead critical tool forces a replan.

4.3 Interpretation. Both scenarios are synthetic and were authored by the system's developers. Offline mode replays recorded model responses, so it demonstrates deterministic policy behaviour of the engine, not model competence. These scenarios are existence proofs and motivate, but do not substitute for, the evaluation in Section 6.

5. Implementation Notes

The system is written in Python 3.11+ on LangGraph, with hosted-model support for Claude and self-hosted inference options. At the time of reading, the repository badge reported 129 passing tests. The repository's own accounting puts the engine at 2,717 lines (28%), the hosting platform at 3,381 (34%), CLI and scripts at 1,539 (16%), own-data plumbing at 1,145 (12%), demo fixtures at 767 (8%) and the local-model client at 326 (3%), about 9,900 lines in total. Entry points include commands to seed demo data, run a goal, list runs, resume a paused run with approval, point the system at a folder of files, and serve a console. The evaluated revision should be pinned by commit hash (the merge of pull request #3 is d259651).

6. Evaluation Protocol (pre-registered; results pending)

6.1 Research questions and hypotheses

- **RQ1 Plan validity.** Does validate-and-re-prompt reduce non-executable plans, and at what cost? H1: no invalid plan reaches the executor by construction; the open question is the number of re-prompts and the semantic quality of valid plans.
- **RQ2 Conflict detection.** How well does literal (subject, metric) matching detect true conflicts compared with normalised, embedding-based and LLM-judge matchers? H2: literal matching has high precision but low recall on formatting-only, scope or unit mismatches and paraphrased metric names.
- **RQ3 Resolution quality.** Does re-querying the least confident claimant outperform simpler resolution rules? H3: it lowers the rate of silently accepted wrong claims relative to highest-confidence-wins, at modest extra cost, and its benefit depends on confidence quality (RQ4).
- **RQ4 Confidence.** Is self-reported confidence informative enough to target re-queries and gate promotion? H4: raw verbalised confidence is overconfident [7]; consistency-based or hybrid estimators improve both calibration and targeting.
- **RQ5 Failure handling.** Which policies matter under which faults? H5: the full policy (retry, degrade, replan) dominates; silent faults (stale or corrupted data) are not fixed by retry and must be caught by reconciliation or validation, in line with [14, 18].
- **RQ6 Memory governance.** Do the human gate, the confidence bar and supersede-only semantics reduce contamination of later runs, and what review burden do they impose? H6: the combined gate yields the lowest downstream error after poisoned or stale inputs, at the highest review cost.
- **RQ7 Consistency and overhead.** How consistent is the system across repeated runs, and what does governance cost in tokens, calls and latency?

6.2 Tasks and data

Three or more domains: the two existing scenarios plus at least one domain built from public tabular and document corpora, so that results do not rest only on author-built fixtures. A seeded scenario generator produces N = [TBD] scenarios per domain with ground truth, covering: true disagreement; stale snapshot; scope or unit mismatch (for example fiscal-year-to-date versus single quarter); formatting-only difference; paraphrased metric names; and non-conflict controls to measure false positives. Fault schedules cover transient errors, permanent outage, timeouts, silent stale data, corrupted output and schema change. Memory scenarios inject a wrong high-confidence claim in run 1 and measure contamination in run 2. The generator, scenarios and traces will be released.

6.3 Baselines and ablations

- **B0:** single ReAct-style agent [23] with all tools.
- **B1:** graph supervisor with handoffs but no reconciliation or gate.
- **B2:** AutoGen group chat [3].
- **B3:** a multi-agent debate baseline [25] for resolution only.
- **AI-OS ablations:** without plan validation; without reconciliation; without retry; without degrade; without replan; without the human gate; with overwrite instead of supersede; with confidence-only or human-only promotion.

6.4 Metrics

- **Task outcome:** end-to-end correctness against ground truth; report completeness; whether the report correctly names missing information.
- **Reconciliation:** precision, recall and F1 of conflict detection; final-claim accuracy; silent-wrong-resolution rate (a wrong claim accepted without human involvement); escalation rate.
- **Confidence:** expected calibration error, Brier score, AUROC for predicting claim correctness, and the probability that the lowest-confidence claimant is the incorrect one.
- **Fault handling:** completion rate per fault type; wasted calls; wall-clock time.
- **Memory:** precision of promoted facts; downstream error after contamination; recall@k and latency at 10^2, 10^3 and 10^4 facts for lexical, embedding and hybrid recall; human items reviewed per run.
- **Consistency and cost:** pass^k over repeated runs [10]; tokens, tool calls and latency per run.

6.5 Models and statistics

At least two hosted models and one open-weight model served locally, to test whether a self-hosted model can hold the schemas the planner and agents require. Temperature and seeds are fixed and reported. Each scenario is run at least [TBD ⩾ 10] times per condition. We report means with 95% bootstrap confidence intervals and use paired tests across conditions. Hypotheses and the analysis plan are fixed before runs. The human approver is simulated by an oracle at scale and complemented by a small human study (n = [TBD]) of review time and accuracy.

6.6 Results

[TBD. Planned outputs: (Table 1) conflict-detection precision/recall by conflict class and matcher; (Table 2) resolution policies compared on accuracy, silent-wrong rate, escalation and cost; (Figure 1) reliability diagram and AUROC for confidence estimators; (Table 3) fault-type by policy completion matrix; (Figure 2) contamination after poisoned input by promotion policy; (Figure 3) pass^k curves versus baselines; (Table 4) overhead.]

7. Discussion

Where the operating-system analogy holds. The system shares several ideas with operating systems: capability-style access control through tool grants, isolation between tenants, a two-tier memory hierarchy, process suspension and resumption through checkpoints, and an audit log. It does not provide what AIOS [1] provides, namely kernel scheduling of LLM requests and context switching across agents. Reviewers familiar with [1] may object to the name; consider framing the contribution as an orchestration runtime or governance layer, or justify the term explicitly.

What the evaluation should settle. The most informative result would be a decomposition of failures by which policy prevents them. If most residual errors are silent semantic ones, the next step is richer claim identity and calibrated confidence, which are the two weaknesses the repository already names.

Security. Ingesting folders of documents creates an injection surface: text in a file can attempt to instruct an agent, and promoted memory can persist a poisoned fact [11]. Named-query defaults, read-only access, grants and the human gate mitigate this, but a threat model and an adversarial evaluation are future work.

AI-assisted development. The commit history credits an AI model as co-author. Many venues require disclosure of AI assistance in code and writing; this applies to this draft as well.

8. Limitations and Threats to Validity

- **Literal conflict identity.** Matching on exact (subject, metric) strings and exact values misses semantically equal claims and non-numeric conflicts. This is the weakest load-bearing part of the design.
- **Uncalibrated confidence.** Self-reported confidence drives re-query targeting, escalation and promotion; it is a heuristic until calibrated against outcomes [7].
- **Lexical recall.** Durable recall is lexical plus recency and confidence, which will not scale to very large fact stores.

- **Unenforced grant fields.** Rate limit, secret reference and maximum wall-clock time are stored but not enforced.
- **Evaluation validity.** Scenarios are synthetic and author-built; the human gate is simulated by an oracle in bulk runs; LLM nondeterminism and model updates can shift results; and findings may not transfer to other domains or to much larger tool sets.
- **Scope.** The system is not a general agent framework; generality across arbitrary domains was deliberately excluded from the first version.

9. Conclusion

AI-OS argues that reconciliation, failure handling and memory promotion deserve first-class, testable policy outside the model. The architecture is documented and its weaknesses are stated. The contribution becomes a research result only when Section 6 is executed; the planned experiments are designed to show which policies earn their cost and where simple literal mechanisms fail.

References

Items marked [verify] were cited from memory or from secondary summaries and must be checked against the original source.

[1] K. Mei, X. Zhu, W. Xu, W. Hua, M. Jin, Z. Li, S. Xu, R. Ye, Y. Ge, Y. Zhang. AIOS: LLM Agent Operating System. arXiv:2403.16971, 2024.

[2] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[3] Q. Wu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.

[4] LangChain. LangGraph, software and documentation. [add official URL and the version used; the repository badge states 1.2]

[5] S. Kim et al. An LLM Compiler for Parallel Function Calling. arXiv:2312.04511. [verify author list]

[6] M. Cemri et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657; NeurIPS 2025.

[7] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, B. Hooi. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. ICLR 2024; arXiv:2306.13063.

[8] S. Kadavath et al. Language Models (Mostly) Know What They Know. arXiv:2207.05221, 2022. [verify]

[9] R. Xu, Z. Qi, Z. Guo, C. Wang, H. Wang, Y. Zhang, W. Xu. Knowledge Conflicts for LLMs: A Survey. EMNLP 2024, pp. 8541–8565.

[10] S. Yao, N. Shinn, P. Razavi, K. Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. ICLR 2025; arXiv:2406.12045.

[11] A survey on the security of long-term memory in LLM agents (attacks, defenses and governance across the memory lifecycle). arXiv:2604.16548, 2026 preprint. [add authors and final title]

[12] Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025. [verify authors]

[13] Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956. [verify id and authors]

[14] When Tools Fail: Benchmarking Dynamic Replanning and Anomaly Recovery in LLM Agents (ToolMaze). arXiv:2606.05806, 2026 preprint.

[15] When Agentic Executions Fail: Detecting and Localizing Runtime Faults from Telemetry (AgentChaosBench). arXiv:2608.14680, 2026 preprint.

[16] Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems. arXiv:2606.01416, 2026 preprint.

[17] Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench. arXiv:2604.16706, 2026 preprint.

[18] Failing Tools: Benchmarking LLM Agent Recovery Under Runtime Tool Failures. OpenReview (ACL ARR 2026 May submission), id j7YsSnA64D; under review.

[19] Agent Zero Memory: Provenance-Aware Long-Term Memory for LLM Agents. arXiv:2608.29606, 2026 preprint.

[20] Evaluation and Benchmarking of LLM Agents: A Survey. arXiv:2507.21504, 2025.

[21] Survey on the intersection of AI and operating systems. arXiv:2407.14567, 2024. [add exact title and authors]

[22] Y. Shen et al. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. arXiv:2303.17580, 2023. [verify]

[23] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.

[24] Memory of Memory (P-Mem). arXiv:2609.25054, 2026 preprint. [add authors]

[25] Y. Du et al. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325, 2023. [verify]

Appendix A. Reproducibility Checklist

- Pin the repository commit (merge of PR #3: d259651, or the final evaluated commit) and the dependency lockfile.
- Report model names, versions, temperature, seeds and any prompt templates; release prompts.
- Release the scenario generator, ground truth, fault schedules and all traces.
- Separate offline-replay runs (engine determinism tests) from live-model runs (evidence about model behaviour).
- Report tokens, calls, latency and monetary cost per condition.
- State the confidence threshold θ, retry limits and backoff settings used.
- State how the human approver was simulated and release the human-study protocol.
- Disclose AI assistance in code and writing according to the target venue's policy.